

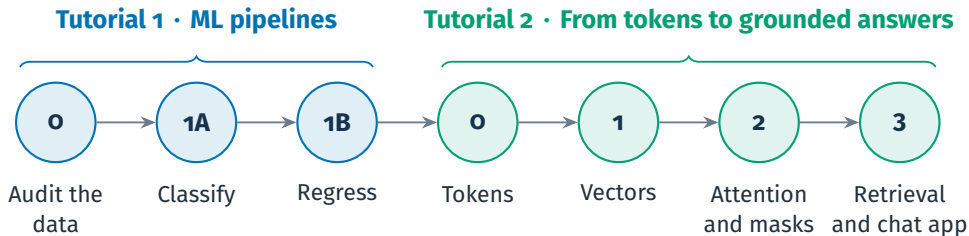
Lab 2

From health data to grounded answers

Tutorial 1 · Machine-learning pipelines with scikit-learn

Tutorial 2 · Tokens, attention, decoding and retrieval-augmented generation

Seven notebooks, one path



Tutorial 1 teaches how to **judge a result**; Tutorial 2 teaches how a language model **produces one**.

Ground rules for both tutorials



Teaching data only

A synthetic CSV, bundled benchmarks, and fictional clinic documents.



Sensitive data stays outside

Patient data, PHI and credentials stay out of every notebook, prompt, screenshot and log.



We study mechanisms

Results show how methods work. Clinical use needs its own evidence.

TUTORIAL 1

Machine-learning pipelines

Audit, split, build, compare, evaluate once — and state the limits of the number.

Two kinds of prediction

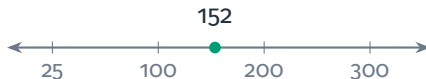
Classification

benign

malignant

The target is a **category**

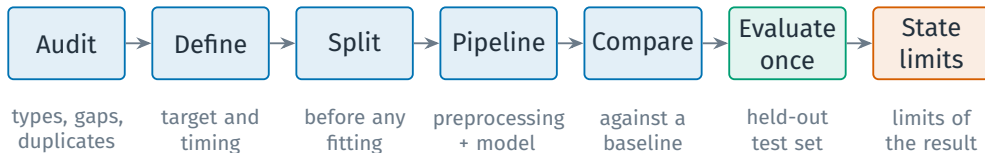
Regression



The target is a **number**

The target type decides the model family, the baseline, the splitter and the metrics.

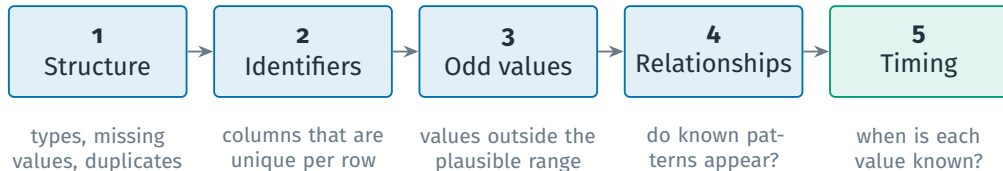
One workflow, every time



Know the data before modelling it

What should you check in any health dataset before anyone builds a model on it?

Five checks before any model



Each check ends in a decision or a question for the data owner. The course CSV is the worked example.

Checks 1 and 2: structure and identifiers

- Report **missing values** per column
- Count **exact duplicate rows** and remove them before splitting
- Count **distinct values**: a column that is unique per row is an identifier
- Keep identifiers out of the features

EXAMPLE · Course CSV · 55,500 rows

0

missing
values

534

duplicate
rows

90%

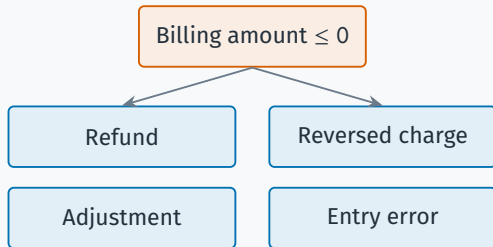
of Name
values are
unique

A model given an identifier memorises rows; zero missing values usually signals synthetic data.

Check 3: treat an odd value as a question

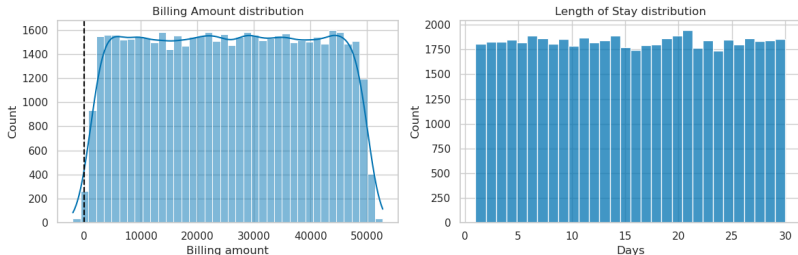
- List values outside the plausible range
- Write down every explanation that fits
- Keep the rows until the data owner confirms one

EXAMPLE · Course CSV · 106 bills at or below zero



Each explanation needs a different response, so keep the rows until one is confirmed.

Plot the same two variables

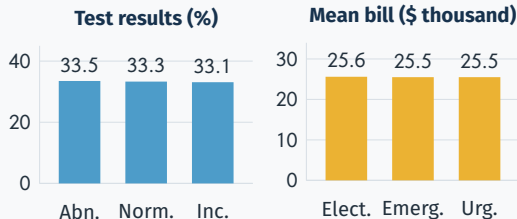


A summary table gives the minimum; the plot shows **how many** values sit near it and whether the shape is believable at all.

Check 4: do known relationships appear?

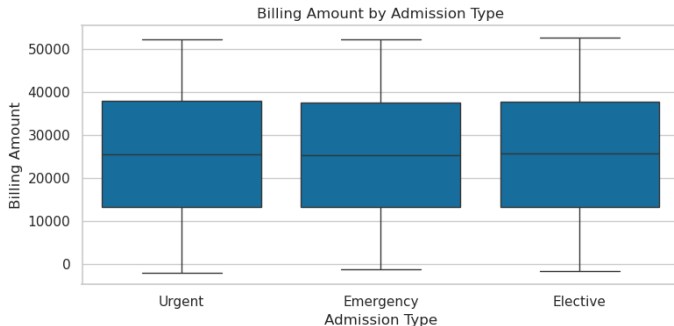
- Pick a relationship you expect clinically
- Plot it
- A flat result means weak signal or generated data

EXAMPLE · Course CSV · expected: emergency care costs more



When expected patterns are absent, use the data for practice and benchmark on other data.

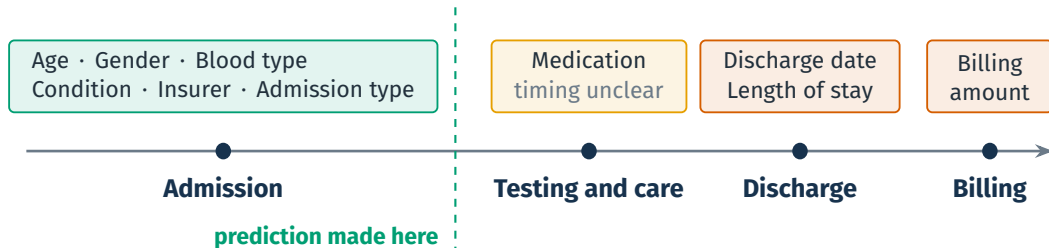
Billing by admission type



Emergency, urgent and elective stays bill almost identically. Real hospital data would differ, so this file cannot teach that relationship.

Check 5: prediction time decides what is usable

EXAMPLE · COURSE CSV, PREDICTING AT ADMISSION



State the moment of prediction first; a column is usable only if its value exists **at that moment**.

Give every column a role

ROLE	DECISION	EXAMPLE · COURSE CSV
Available	use, after confirming timing	Age, Condition, Admission type
Timing unclear	ask the data owner	Medication
Future or outcome	exclude from features	Discharge date, Billing amount
Identifier	exclude from features	Name
Operational	exclude, or state the site dependence	Doctor, Hospital, Room

Write the role of every column down before choosing features.

Leakage takes two forms

1 • FUTURE INFORMATION

Using a value from the future

Predicting at admission with Discharge Date, or with a length of stay derived from it.

2 • PREPROCESSING

Letting a fitted step see every row

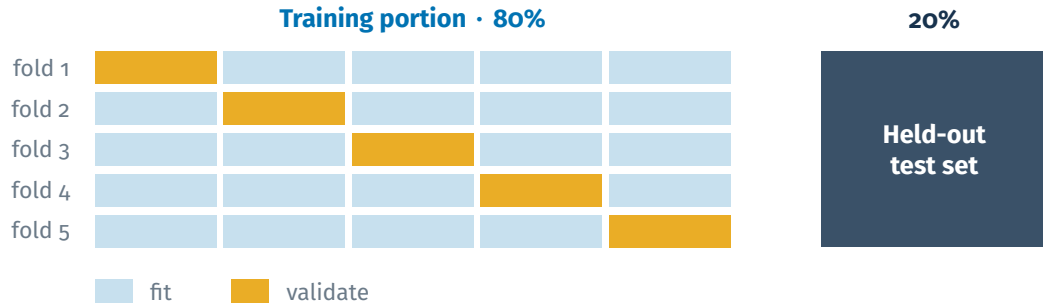
Scaling, imputing, encoding or selecting features before the data are split.

Leakage runs silently and makes the result look **better**.

Predicting a category

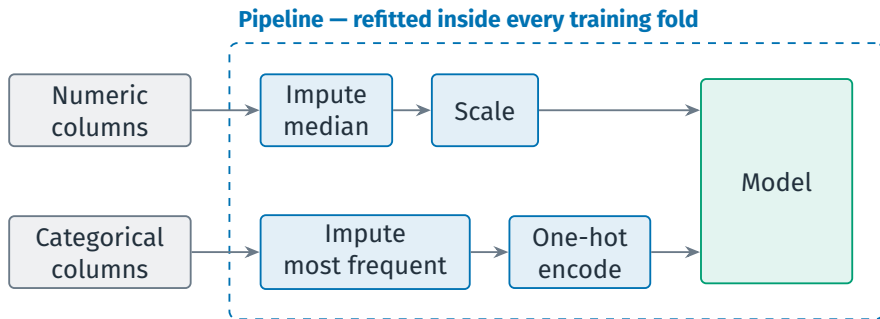
Can tumour measurements classify a recorded diagnosis — and how would we know?

Split first. Evaluate once.



Cross-validation picks the candidate; the test set grades it **once**.

Everything that learns goes inside the pipeline



Every step that learns from data belongs inside the pipeline.

Encode each category as its own column

Admission type		Emerg.	Elect.	Urgent
Emergency		1	0	0
Elective	→	0	1	0
Urgent		0	0	1

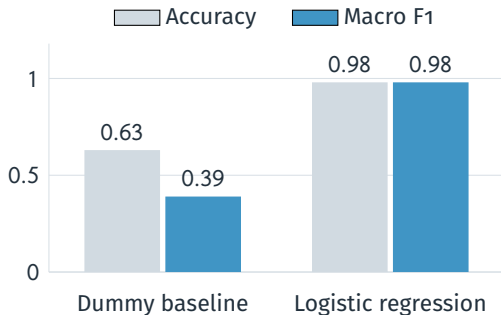
Codes 1, 2, 3 imply an order

They tell the model the categories are ranked and evenly spaced.

6 columns → 25

Doctor has 40,000 values: one-hot would add 40,000 columns.

The baseline sets the bar



Classes are unbalanced:
63% benign, 37% malignant.

A dummy that always answers
“benign” scores **0.63 accuracy** and
misses every malignant case.

Choose the metric **before** seeing the results.

Read errors by direction

		Predicted	
		malignant	benign
Actual	malignant	41	1
	benign	1	71

False negative

A malignancy the model missed.

False positive

An unnecessary follow-up.

Accuracy weighs both equally. For a patient, a missed malignancy is far worse.

Accuracy 0.98 on 114 held-out cases is one estimate from one split.

Which mistake costs more?

PRECISION

Of the cases flagged, how many were real?

$$\frac{TP}{TP + FP}$$

Lead with it when a **false alarm** is costly: an invasive follow-up, alert fatigue.

RECALL · SENSITIVITY

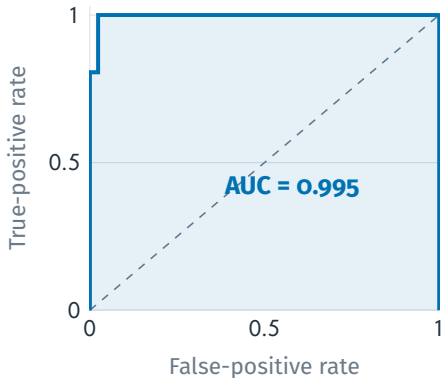
Of the real cases, how many were found?

$$\frac{TP}{TP + FN}$$

Lead with it when a **miss** is costly: a malignancy, a deteriorating patient.

Set the threshold by the cost of each mistake.

AUC measures ranking



What it asks

Draw one case of each class. How often does the malignant one score higher?

What you still need

- a chosen decision threshold
- a calibration check
- evidence that acting on it helps

Predicting a number

Can baseline measurements estimate a disease-progression score — and by how much do we miss?

Three metrics, three questions

MAE

How large is a typical error?

$$\frac{1}{n} \sum_i |y_i - \hat{y}_i|$$

In the target's units. Easy to explain.

RMSE

Same, but punish big misses

$$\sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$$

A 10-unit error counts four times a 5-unit error.

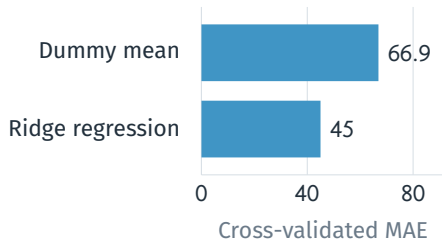
R²

How much better than the mean?

$$1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Unitless. Zero means equal to predicting the mean.

Better than baseline — and still mostly unexplained



0.48

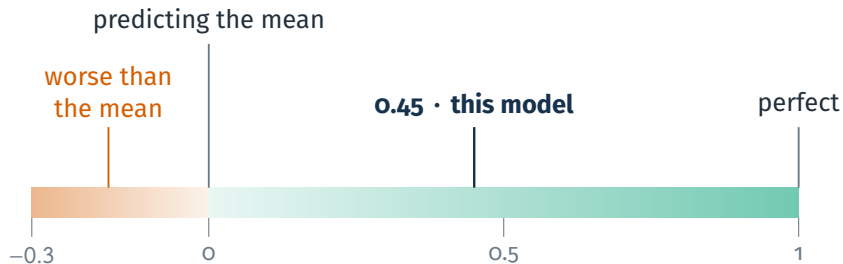
R^2 , cross-validated

0.45

R^2 , held-out test

A model can beat its baseline and **still be too weak to act on.**

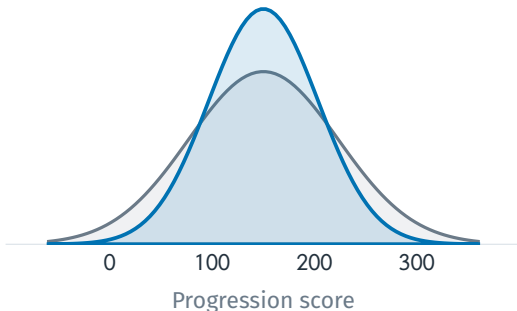
Reading R^2 correctly



A negative R^2 means the model did **worse than predicting the mean**.

What the metrics hide: compressed predictions

Actual values (SD 73) Predictions (SD 54)

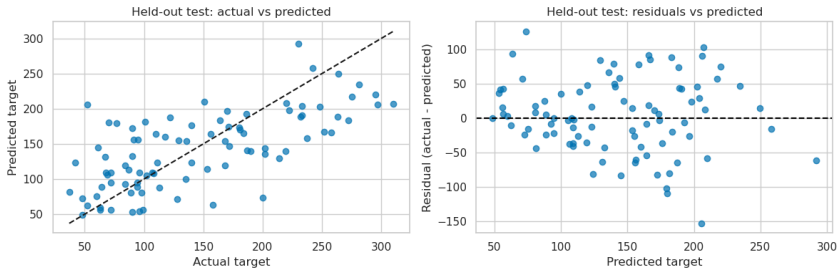


The model rarely predicts very high or very low scores, so it is least informative where decisions are made.

Only the plot reveals this.

Illustrative curves drawn from the measured standard deviations.

Actual against predicted, and the residuals



Predictions bunch toward the middle: high scores are under-predicted and low scores over-predicted, which no single metric reports.

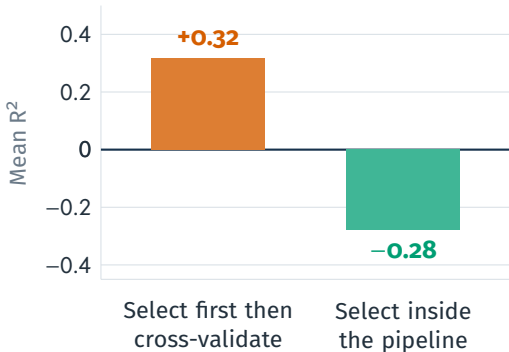
Leakage can manufacture a result

THE SET-UP

200 rows, 2,000 features and a target, **all pure random noise**.

Pick the 20 features that correlate best with the target, then cross-validate a model.

The true answer is $R^2 = 0$.



The defence is structural

LEAKS

Select features
on **all** rows

Split into folds

Fit and score

SAFE

Split into folds

Select features
on the **training fold**

Fit and score

one Pipeline object

Put feature selection inside the pipeline, so it only ever sees the training fold.

Before you report any number

- ✓ Every feature exists at prediction time
- ✓ The split came before any fitting
- ✓ Every fitted step is inside the pipeline
- ✓ The metric was chosen before the results
- ✓ The model was compared with a dummy baseline
- ✓ The test set was used exactly once

If a result is much better than expected, **check for leakage first.**

What still needs separate evidence



Another site

or a later year



Calibration

are 80% risks right
80% of the time?



Subgroups

who carries the
errors?



Causation

needs a causal
study design



Usefulness

does acting on it
help?

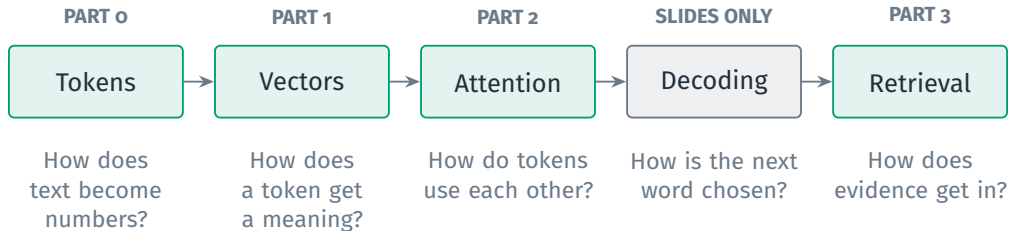
A high score shows the workflow was executed correctly.

TUTORIAL 2

From tokens to grounded answers

How text becomes numbers, how a model combines them, how it picks words — and how evidence gets in.

Five mechanisms, five questions



A model reads tokens

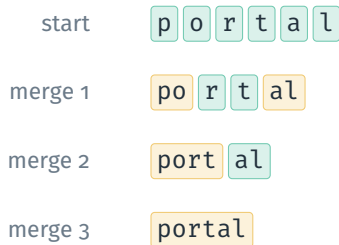
How does a sentence of clinical text become the numbers a language model actually receives?

Clinical words arrive in pieces

WRITTEN	WHAT THE MODEL RECEIVES	TOKENS
portal	portal	1
metformin	met form in	3
myocardial infarction	myocard ial inf ar ction	5
HbA1c	H b A 1 c	5

Clinical vocabulary costs several tokens per word, so the model handles it as **fragments**.
Tokenizer: `cl100k_base`.

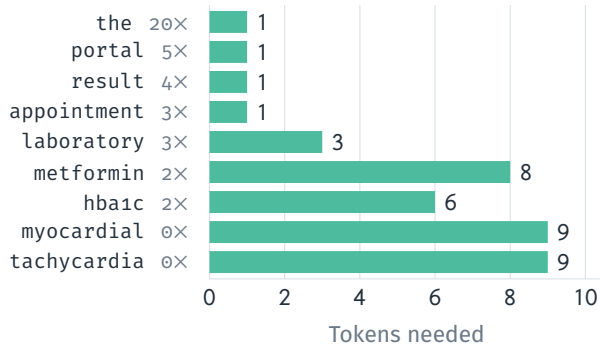
Byte pair encoding: merge what is frequent



Illustrative merge order. Gold marks the newly merged symbol.

BPE repeatedly merges the most frequent adjacent pair; the ordered list of merges **is** the tokenizer.

Frequency decides the cost

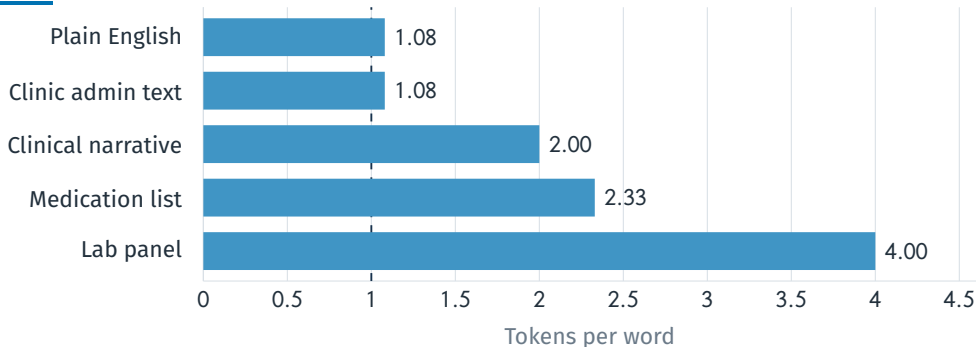


A hand-built tokenizer, trained on 14 fictional clinic sentences.

Grey: times the word appeared in training.

Frequency decides token cost: rare words cost more.

Clinical text costs more



Price and context limits are counted in **tokens**, and clinical text uses two to four times more.

Same meaning, different tokens

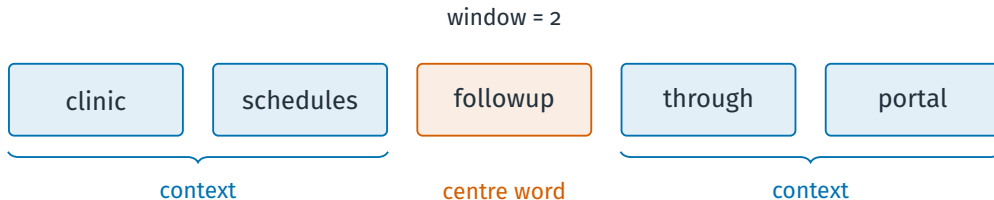
A CLINICIAN SAYS		... OR SAYS		SHARED TOKENS
laboratory result	↔	bloodwork		0
appointment	↔	visit		0
laboratory result	↔	lab result		1
proxy access	↔	caregiver access		1

Tokens come from character frequency, so meaning has to come from vectors (next) or a **terminology** (Part 3).

From identity to meaning

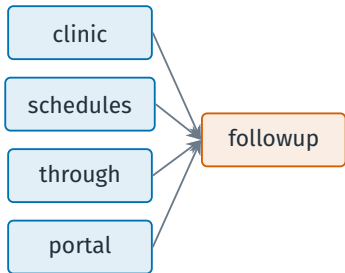
A token ID is only a row number. How does a token acquire a meaning?

Meaning from neighbours



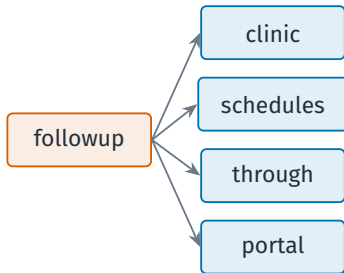
Words used in similar contexts end up with similar vectors — **for this corpus only.**

Two training directions



CBOW

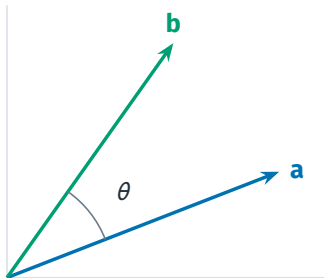
context predicts the centre



Skip-gram (used here)

the centre predicts its context

Comparing vectors: cosine similarity



$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

- Compares **direction**, ignores length
- 1: same direction 0: unrelated
- Measured in the original 50 dimensions

A high similarity describes how two words are used in one corpus.

Variety of context beats more rows

TEMPLATE CORPUS

0.75

median similarity between selected words

240 sentences from one repeated pattern.
Nearly every word looks like every other.

VARIED CORPUS

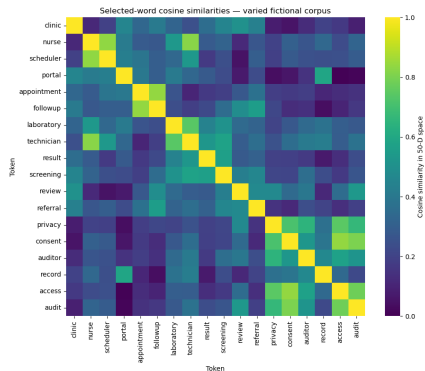
0.28

median similarity between selected words

364 sentences with different structures.
Words separate into usable
neighbourhoods.

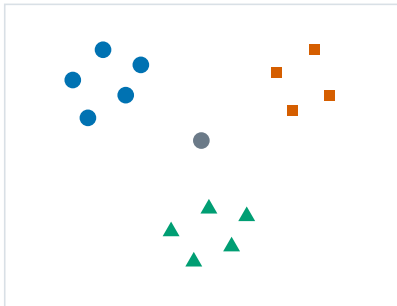
Embedding quality comes from **variety of context**.

Similarity in the varied corpus



Read one row: words used in similar sentences score high, and the block structure is what retrieval will rely on.

Reading a t-SNE map



Schematic of a t-SNE map.

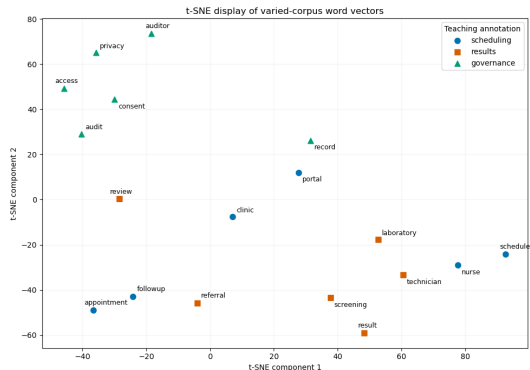
✓ Reasonable

Which words sit in the same local neighbourhood, for this one run.

⚠ Artefacts of the method

- axis directions
- distances between far-apart groups
- the size of a cluster
- the number of clusters

The map from one run



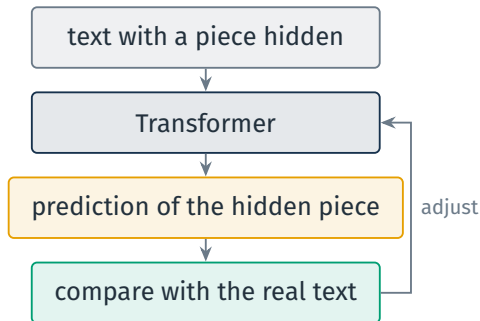
Trust the **local neighbourhoods**; a different seed moves the groups and changes the distances between them.

How tokens use each other

How does one attention operation support two different ways of training a language model?

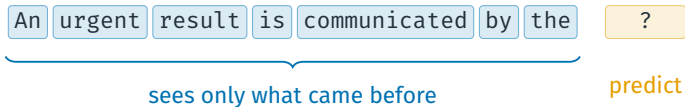
The problem: learning language from raw text

- Labelled health text is scarce and expensive
- Unlabelled text is almost unlimited
- So let the text **label itself**: hide part of it, and train the model to predict what was hidden

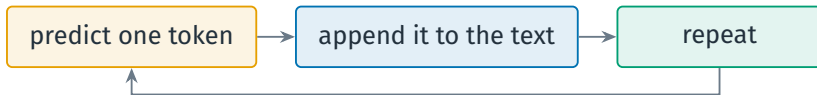


This is **self-supervised learning**: the text supplies its own labels.

Autoregression: predict the next token

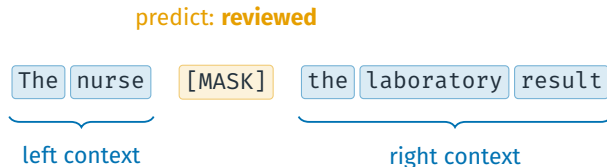


$$P(x_1, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_1, \dots, x_{t-1})$$



GPT-style models **generate** this way: one token at a time, left to right.

Masked language modelling: fill in the blank



- Hide a random share of tokens (about 15% in BERT) anywhere in the text
- Predict each one using the words on **both** sides

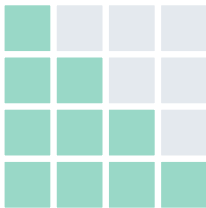
These models **encode** text into representations for classifying and searching.

Same goal, different strengths

	AUTOREGRESSIVE	MASKED LANGUAGE MODEL
Hides	the next token	random tokens anywhere
May look at	earlier tokens only	tokens on both sides
Natural use	writing: answers, summaries, drafts	reading: classifying notes, coding, search embeddings
Families	GPT-style decoders	BERT-style encoders

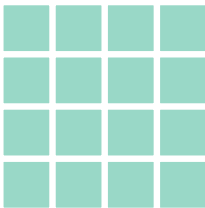
Both learn what words mean **in context** from unlabelled text; Part 2 trains one small model each way.

Both depend on one operation



Autoregressive

each position sees the past



Masked

each position sees everything

To predict a hidden token, a position must **gather information from the positions it is allowed to see**.

Row = the predicting position;
green = positions it may use.

That gathering step is **attention**; the two training styles differ only in which positions are visible.

Attention ends in a weighted average

$$\begin{array}{ccccccc} \mathbf{0.75} & \times & \boxed{[10, 0]} & + & \mathbf{0.25} & \times & \boxed{[0, 20]} & = & \boxed{[7.5, 5.0]} \\ \text{weight} & & \text{value 1} & & \text{weight} & & \text{value 2} & & \text{output} \end{array}$$

Attention **blends** the values in proportion to their weights.

Three roles: query, key, value



Query

What this position is looking for.



Key

How each position can be matched.



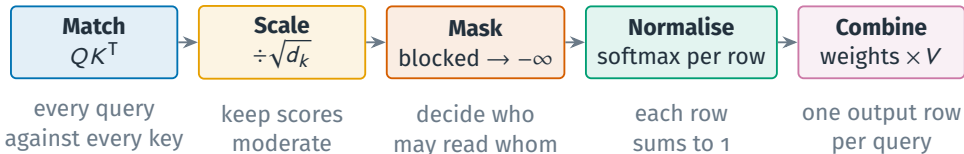
Value

The information each position contributes.

Queries are compared with keys to get weights, which are then applied to the values.

Scaled dot-product attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$



Why divide by $\sqrt{d_k}$?

MODERATE SCORES · [2, 0]



Both keys still contribute.

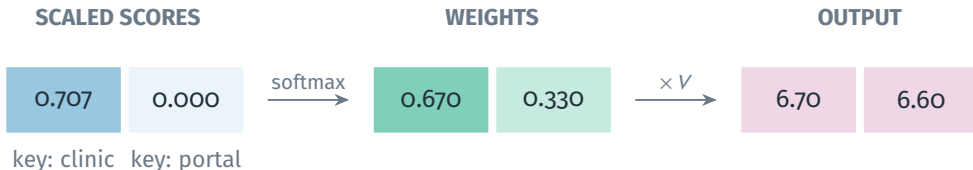
EXTREME SCORES · [8, 0]



Softmax has collapsed onto one key.

Dividing by $\sqrt{d_k}$ keeps scores moderate, so attention stays a blend across keys.

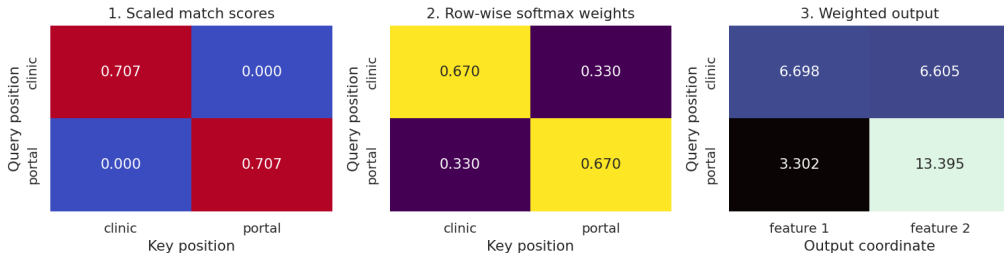
One row, slowly



$$0.670 \times [10, 0] + 0.330 \times [0, 20] = [6.70, 6.60]$$

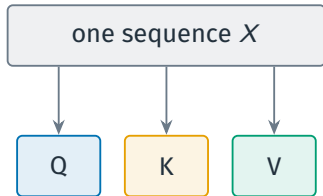
A value's contribution depends on its weight and its size together.

Scores, weights, output



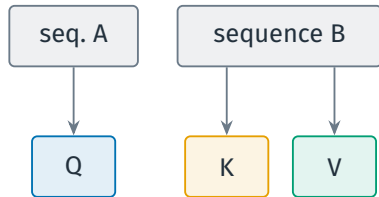
Five lines of code produce all three tables, and the notebook checks them against PyTorch's own attention function.

Self-attention and cross-attention



Self-attention

Q, K and V all come from the same sequence.



Cross-attention

Queries from one sequence, keys and values from another.

Beyond the notebook: attention is blind to word order

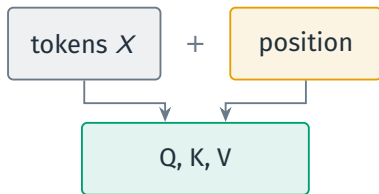
Same tokens, opposite meaning:

result normal not abnormal
result abnormal not normal

Shuffle the input and the outputs are simply shuffled. Position has to be **added to the input**.

0.00

output change after a shuffle,
token content only



0.89

output change after a shuffle,
with positional encoding

A causal mask hides the future

	key position			
	1	2	3	4
1		$-\infty$	$-\infty$	$-\infty$
2			$-\infty$	$-\infty$
3				$-\infty$
4				

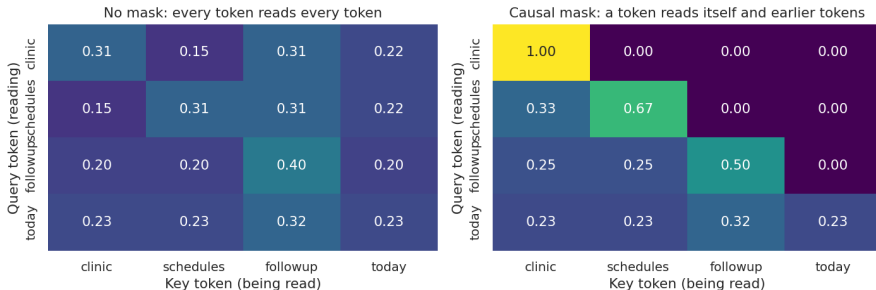
- A position may use itself and earlier positions
- Later positions are set to $-\infty$ **before** softmax
- Their weight becomes exactly 0

First row of the worked example:

$[0.670, 0.330] \rightarrow [1, 0]$

This is the **autoregressive** rule, enforced inside attention: each position sees only itself and the past.

Same Q, K and V; only the mask differs



Blocked cells become exactly 0 and every row still sums to 1: the weight moves to the positions that remain visible.

A mask is a table of who may read whom

PATTERN	RULE FOR EACH ROW	WHERE IT IS USED
Bidirectional	read every token	BERT-style encoders
Causal	read yourself and earlier tokens	GPT-style decoders, chat
Padding	nobody reads [PAD] filler	batches of unequal-length texts
Prefix	prompt both ways, then causal	encoder-decoder models (T5)
Sliding window	read only the last w tokens	long-context models

The arithmetic never changes. Model families differ in **which cells are open**.

Six masks on one sentence

Green = the row token may read the column token



Full attention on n tokens costs n^2 comparisons; a window of width w costs about $n \times w$.

Each training objective is a mask plus a target

AUTOREGRESSIVE

input the patient reports cough

target patient reports cough so

- Target: the input shifted by one
- **Every** position is scored
- Mask: **causal**, or position i reads its own answer at $i + 1$

MASKED LANGUAGE MODEL

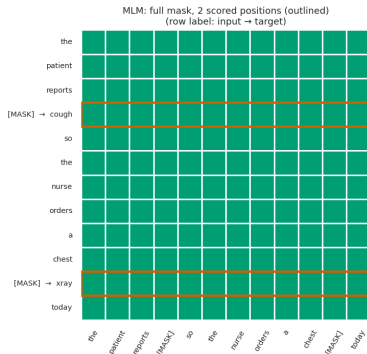
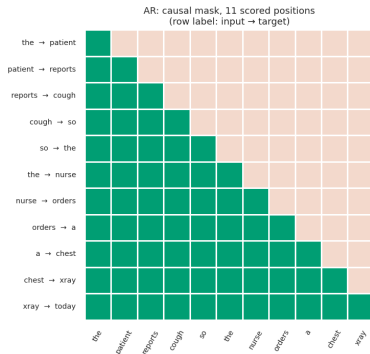
input the patient reports [MASK]

target - - - cough

- Target: the original token at each [MASK]
- About **15%** of positions are scored
- Mask: **full**, because clues sit on both sides

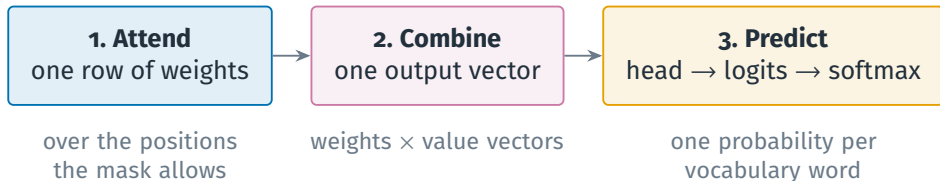
“Mask” has two meanings: the **attention mask** is a table of permissions; the [MASK] **token** is a change to the input.

Both objectives on one sentence



AR scores 11 of 11 positions and MLM scores 2 of 12, but the [MASK] row may read chest xray to its right.

From an attention row to a prediction

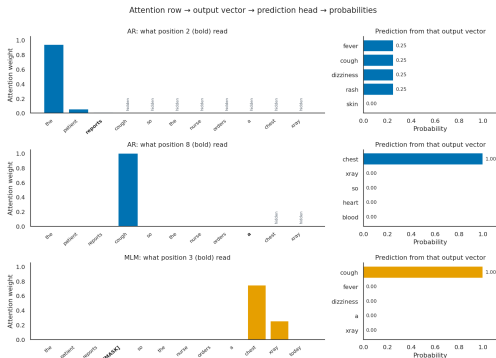


AR: the output at position i predicts token $i + 1$.

MLM: the output at a [MASK] predicts the hidden token.

The **prediction head** is one linear layer from the output vector to a score for every word in the vocabulary.

One architecture, trained two ways



AR cannot know the symptom in advance (0.25 each) yet uses it later to predict the test; MLM reads the test to recover the symptom.

Testing the mask: can the future leak?

Edit two **late** tokens and measure how far the scores at **earlier** positions move.

... orders a chest xray today

... orders a blood test today

- AR: no earlier position can change, by construction
- MLM: earlier positions are **meant** to change

0.0

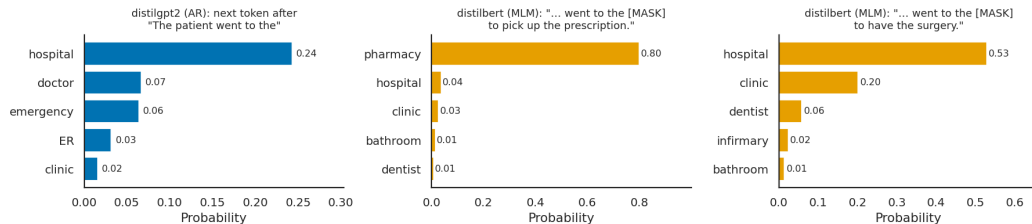
largest earlier change,
AR with causal mask

12.5

largest earlier change,
MLM with full mask

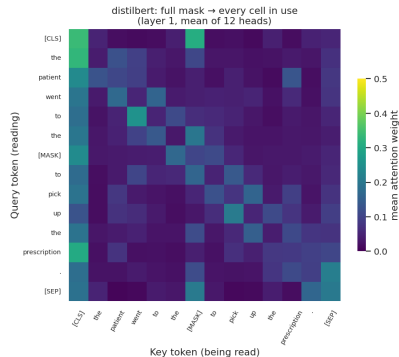
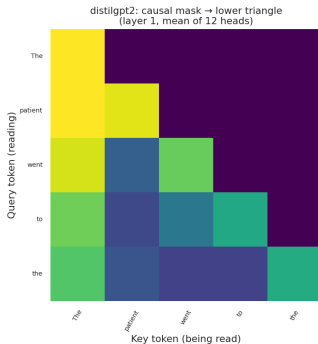
An AR model with a non-zero result here has been reading its own answers: the sequence version of **data leakage**.

The same contrast in pretrained models



The left context is identical in all three panels; DistilBERT's answer moves from pharmacy to hospital with the words on the right.

The masks are visible in real attention maps

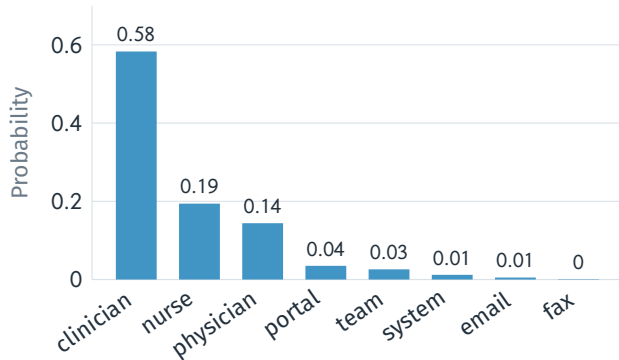


distilgpt2 puts exactly 0 above the diagonal; distilbert puts 38% of its weight there. General-purpose models, not clinical ones.

How the next word is chosen

Why does the same model, asked the same question, give a different answer each time?

The model outputs a score for every token

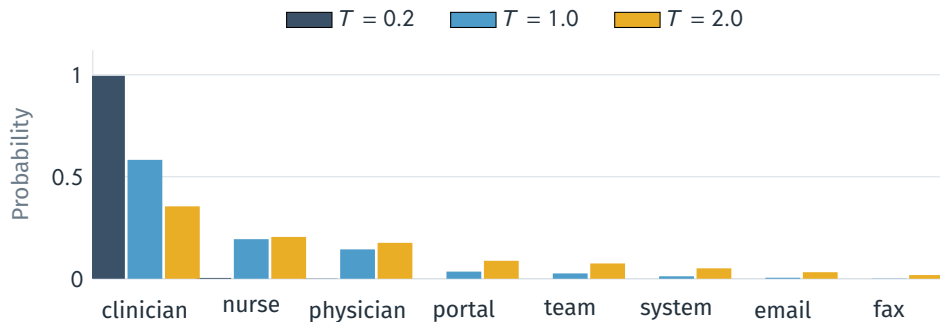


“An urgent result is communicated by the ____”

Softmax turns the model's scores into probabilities.

Decoding picks one, using a rule that you set.

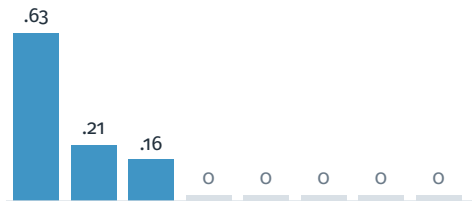
Temperature reshapes the same scores



Lower temperature buys **repeatability**; the model's scores stay the same.

Top-k and top-p remove candidates

TOP-K · KEEP A FIXED NUMBER



$k = 3$: the rest become exactly zero

TOP-P · KEEP ENOUGH TO REACH p

Distribution	Kept at $p = 0.9$
Confident ($\mathcal{T} = 0.5$)	2
Model's own ($\mathcal{T} = 1.0$)	3
Uncertain ($\mathcal{T} = 2.0$)	6

Top-k keeps a fixed count; top-p adapts to the model's confidence.

Same request, five runs

Greedy ($T \rightarrow 0$)



1 distinct output

Sampling ($T = 1.0$)



5 distinct outputs

Different summaries of the same note come from **sampling**, so judge each against the source text.

Choosing settings for health work

TASK	START FROM	BECAUSE
Extracting a value	$T = 0$	one right answer, every run
Answering from retrieved policy	$T = 0$ to 0.3	wording follows the evidence
Drafting text a person will edit	$T \approx 0.7$, top-p 0.9	every output gets reviewed
Anything that will be audited	$T = 0$, version pinned	must be reproducible

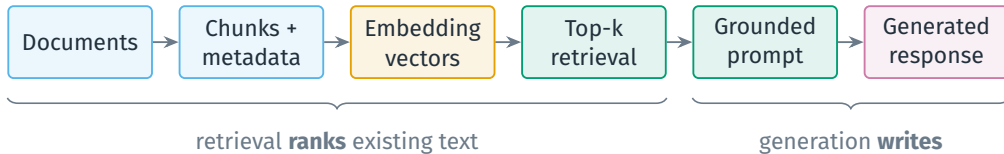
Decoding chooses among what the model already has; new evidence enters through retrieval.

PART 3 • RETRIEVAL-AUGMENTED GENERATION

Getting evidence in

What changes when the same model receives retrieved evidence — and how do we check what it did with it?

The RAG pipeline



RAG changes what is in the prompt; the model's weights stay the same.

Same model, two conditions

DIRECT



RAG



RAG adds three rules: **use only the context, cite the chunk, abstain if the answer is absent.**

Chunking decides what can be retrieved

By heading

Portal registration

Appointment changes

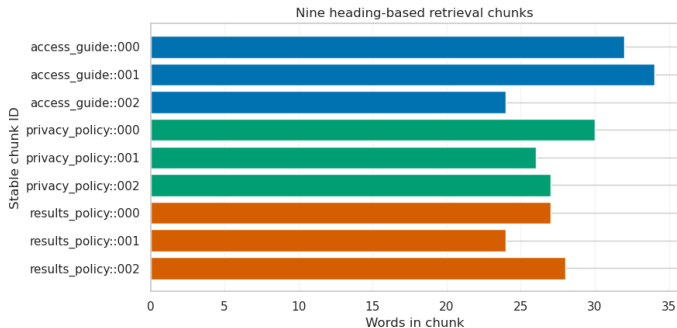
Accessibility

Fixed window



Test chunking strategies on your own questions.

Nine chunks, one per heading



Each chunk has a **stable ID**; every retrieval score, citation and audit later in the notebook refers to these nine IDs.

Four questions, four behaviours to test

CASE	QUESTION	A GOOD SYSTEM ...
Single chunk	How long is a registration code valid?	cites one passage
Multi chunk	Who sends urgent results? Who interprets?	combines two
Unanswerable	What parking fee does the clinic charge?	says it is absent
Partial evidence	Can a parent see a teenager's results?	names the gap

Relevant but incomplete evidence invites the model to fill the gap.

Two ways to score a passage

LEXICAL · SHARED WORDS

$$\frac{|\text{query words} \cap \text{chunk words}|}{|\text{query words} \cup \text{chunk words}|}$$

- Every score can be explained by pointing at words
- Free, local and instant
- bloodwork and laboratory result share nothing

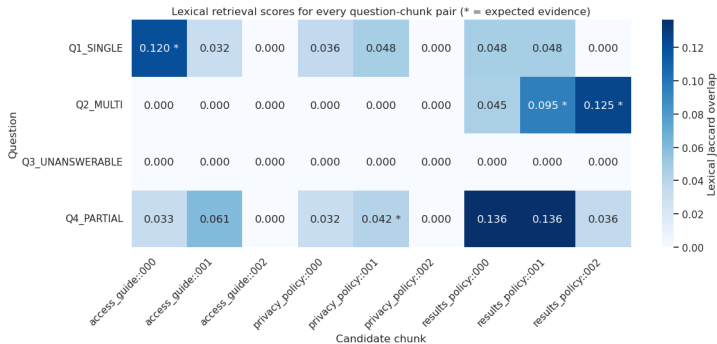
EMBEDDING · SIMILAR MEANING

$$\cos \theta = \frac{\mathbf{q} \cdot \mathbf{c}}{\|\mathbf{q}\| \|\mathbf{c}\|}$$

- Matches paraphrases and synonyms
- Needs an embedding model, often a paid service
- The score is a ranking signal with no stated reason

The notebook starts with the transparent method so that every retrieval decision can be inspected, then adds embeddings.

Every question against every chunk



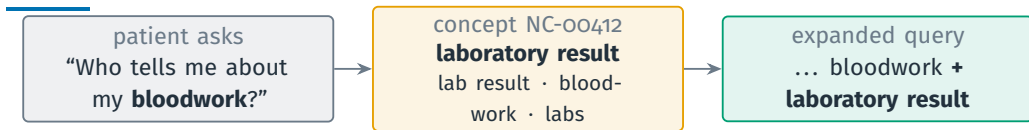
The unanswerable row is all zeros, and top-k would still return four chunks. The partial question's expected chunk scores 0.042, below three off-topic ones.

Top-k always returns k



Top-k means **highest ranked**; relevance is a separate check.

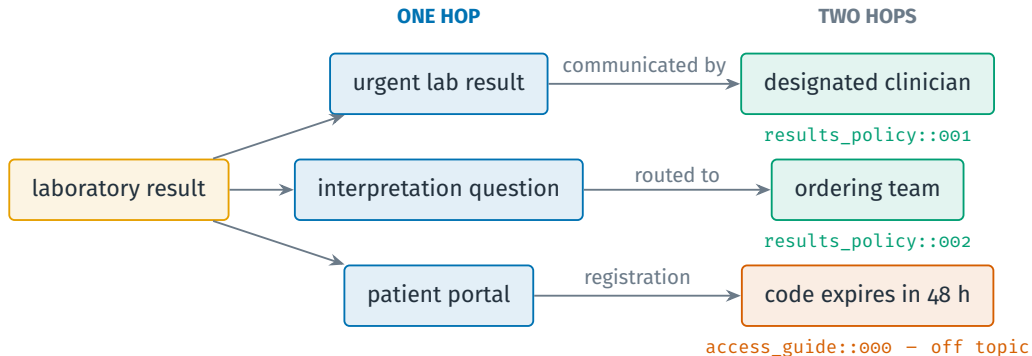
A terminology connects patient words to document words



CHUNK	SCORE BEFORE	SCORE AFTER
Routine results	0.000	0.125
Questions about results	0.037	0.107
Urgent results	0.000	0.038

A terminology states explicitly that two phrases name one concept. Codes here are fictional; real systems use SNOMED CT, LOINC or ICD-10.

A knowledge graph adds relationships – and provenance



Every edge carries its source chunk, so a reviewer can trace why a passage was shown.

What the evidence changed

DIRECT · QUESTION ONLY

“...registration codes might remain valid for anywhere from a few hours to several days. Without details about the particular portal, it’s uncertain...”

169 tokens

RAG · RETRIEVED CONTEXT

“A portal registration code remains valid for **48 hours**. [access_guide::000]”

312 tokens

Recorded gpt-4.1-mini responses, 19 Sept 2026. Q1: “How long does a portal registration code remain valid?”

Retrieved evidence made the answer specific and traceable, at about twice the token cost.

Evaluate in two gates

1 · Retrieval gate

Did the retrieved set contain every chunk the answer needs?

fails → fix chunking, top-k, query

2 · Generation gate

Does the answer cite only retrieved chunks, and abstain when evidence is missing?

fails → fix the prompt, check the model

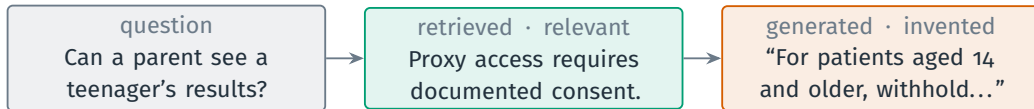
A correct chunk with an invented fee is a **generation** failure.

A citation is a pointer for a human check

HAND-WRITTEN ANSWER	WHAT IS WRONG	AUTO CHECK
"...48 hours. [access_guide::000]"	correct	pass
"...and a \$15 fee . [access_guide::000]"	fee is invented	pass
"...48 hours. [privacy_policy::000]"	cites the wrong chunk	pass
"Yes... aged 14 and older , withhold..."	policy is invented	pass
"...72 hours. [access_guide::007]"	chunk ID is fabricated	flagged

The checker verifies the citation label; a person verifies the claim.

The partial-evidence trap

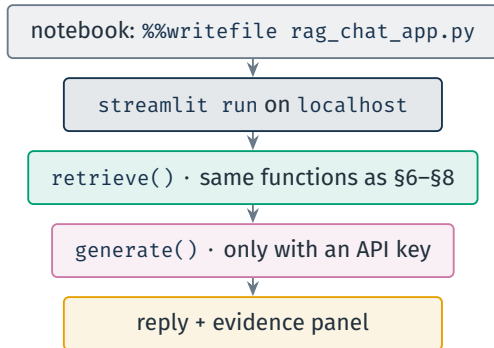


The corpus is silent on minors,
ages and sensitive results.

The safe answer covers the supported part and **names the gap**.

From notebook to chat interface

- **Streamlit** turns a Python script into a web page
- The script **reruns top to bottom** on every interaction
- `st.session_state` keeps the conversation
- `@st.cache_data` keeps the corpus and embeddings
- `st.chat_input` and `st.chat_message` draw the chat box



The interface adds no intelligence. It gives someone else a way to use what you built.

The chat app, asked a patient's question

EXAMPLE · retrieval-only mode, no key, no cost

You Who tells me about my bloodwork if something is wrong?

App No model was called. The best-matching passage is **[results_policy::000]** (Routine results).

Terminology added: **laboratory result results**

EVIDENCE PANEL

#	chunk	score
1	results_policy::000	0.125
2	results_policy::002	0.107
3	results_policy::001	0.038
4	access_guide::000	0.000

Tested with AppTest; runs on
localhost:8501.

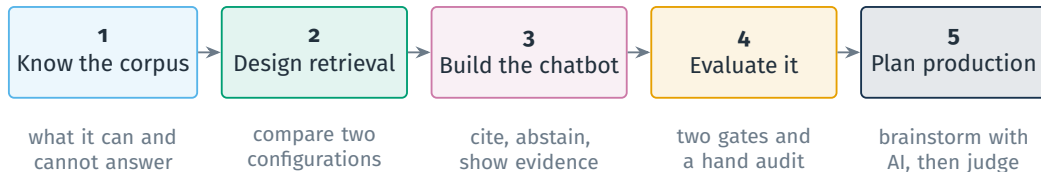
Rank 4 scored 0.000 and was still shown. A chat box looks finished, so **put the limits on the screen.**

Limits to carry forward

- Nearest neighbours still need a relevance check
- Retrieved sources can be incomplete or wrong
- A cited answer still needs its claim checked
- The terminology and graph are tiny, fictional and hand-built
- Replayed responses are one run on one date
- Live services add cost, privacy and retention questions

Treat retrieved text as **data to check**.

Your turn: Lab 2



Every step has a model in **Part 3**. Add an **AI use record** and a **one-page reflection**.

Write your test questions **before** you tune the system, including ones the corpus cannot answer.

Seven things to keep

TUTORIAL 1

1. Compare with a **baseline** before anything else.
2. Every step that learns goes **inside the pipeline**.
3. A surprisingly good result means: **look for leakage**.

TUTORIAL 2

4. Clinical text is **expensive in tokens**.
5. The **mask** decides what a model may read.
6. Temperature o buys **repeatability**.
7. A person still has to **check each cited claim**.